

Language Paradigms

- **Imperative Programming**

IDEA: List of commands

EXAMPLES: C, C++, Java, Pascal, Ada,
FORTRAN, COBOL

- **Functional Programming**

IDEA: Functions and function composition

EXAMPLES: LISP, Scheme, ML, Haskell

- **Logic Programming**

IDEA: Symbolic logic

(Assert some facts, then deduce new facts.)

EXAMPLE: Prolog

LISP/Scheme Outline

1. Functional Programming
2. Brief history
3. The Scheme Interpreter
(Calling Scheme functions)
4. Data in Scheme
5. Defining new Scheme functions

Functional Programming

Much mathematical notation uses functions and function composition.

Function examples:

$\sin(x)$ *Prefix notation*

$\ln(x)$

$-x$

$f(x, y)$

$x + y$ *Infix notation*

$x - y$

$|x|$ *Various other notations*

Function composition examples:

$(x + y) * 4$

$(\ln(x) + \sqrt{y})^2$

Scheme and other LISP dialects use **prefix notation**, with this form:

(functor arg1 arg2 arg3 ...)

Examples:

Scheme form	Math equivalent
(sin x)	$\sin(x)$
(sqrt x)	$\sqrt{x}$
(+ x y)	$x + y$
(- x y)	$x - y$
(* x y)	$x * y$
(/ x y)	x / y

Functions can be **composed** in Scheme, as they are in math.

Examples:

Scheme form

Math equivalent

(sin (+ x y))

$\sin(x + y)$

(+ (sqrt x) (sqrt y))

$\sqrt{x} + \sqrt{y}$

(/ (+ x 1) (- y 2))

$\frac{x + 1}{y - 2}$

Scheme form

Math equivalent

(* (+ x 20)
 (sqrt y))

(- (+ (- y 5) 3)
 (* 15 (+ x 2)))

$$\frac{x + \sqrt{y}}{2}$$

$$3 (x + y) - 5 z$$

Historical Background

Mathematical prelude:

Lambda calculus

Alonzo Church — 1940s

Model of computation based on functions

Later shown equivalent in power to
Turing machines

Notation:

$$\lambda x y (x + y)$$
$$\lambda x (2 * x * x)$$

LISP:

John McCarthy

Designed for artificial intelligence

First version completed in 1959 at MIT

LISt Processing

Current uses include:

- Artificial Intelligence applications
- Emacs editor

Using the Scheme48 interpreter

elvis % **scheme48**

Welcome to Scheme 48 0.57 (made by capuano on Tue Aug 6 09:31:41 EDT 2002).
Copyright (c) 1993-2001 by Richard Kelsey and Jonathan Rees.
Please report bugs to scheme-48-bugs@martigny.ai.mit.edu.
Type ,? (comma question-mark) for help.

> **(+ 3 5)**

8

> **(- (+ 3 5) 2)**

6

> **(define (add3 x) (+ x 3))**

; no values returned

> **(add3 10)**

13

> **,exit**

elvis %

Quote

Quote: Returns argument unevaluated

```
> (+ 3 5)
```

```
8
```

```
> (quote (+ 3 5))
```

```
(+ 3 5)
```

```
> (quote +)
```

```
+
```

Often abbreviated as ' with no parentheses:

```
> '(+ 3 5)
```

```
(+ 3 5)
```

```
> '+
```

```
+
```

Data in Scheme

ATOMS

Numbers

Integers: 2, -35, 0

Often, arbitrary precision integers:
123456789123456789123456789

Reals: 3.14, -12.5

Often, scientific notation: 6.02e23

Rationals: 2/3, -1/6

Complex numbers: 4+3i

Strings: "apple", "25"

Symbols: a, apple, x5\$3B#y

LISTS

(a b c)

(apple)

The empty list: ()

Nested lists: (aardvark (bat cat) ((dog)))

Mixed types: (12 (a b c) "Hello" zebra)

Two special symbols

Symbol	Meaning
#T	true
#F	false

List operators: car, cdr

car: Returns the first element of a list

cdr: Returns all of a list except its first element

Examples:

Expression	Return value
(car '(a b c d))	a
(cdr '(a b c d))	(b c d)
(car '((a b) c (a b)))	(a b)
(cdr '((a b) c (a b)))	(c (a b))
(car '(a))	a
(cdr '(a))	()

Examples: Using car and cdr together

```
(car (cdr '(a b c)))
```

```
(car (cdr (cdr (cdr '(a b c d e)))))
```

```
(cdr (car (cdr '((a b) (c d) (e f)))))
```

A shorthand for car/cdr

Short form

(cadr x)
(cdaddr x)

Long form

(car (cdr x))
(cdr (car (cdr (cdr x))))

Examples:

(cadr '(x y z))

(caar '((w x) (y z)))

(cddr '((w x) (y z)))

(caddr '(1 3 5 7))

More car/cdr examples

1. Write an expression using **car**, **cdr**, and **((a) (b) (c))** that returns **(c)**.
2. Write an expression using **car**, **cdr**, and **((a) (b) (c))** that returns **a**.

List operators: cons

(cons *x* *y*)

Returns the list whose car is *x* and whose cdr is *y*.

Examples:

Expression

Return value

(cons 'a '(b c d))

(a b c d)

(cons 'a '())

(a)

(cons '(a b) '(c d))

((a b) c d)

Exercises: car, cdr, cons

What is the value of:

`(car (cons 'a '(b c)))`

`(cons (car '(a b c)) (cdr '(d e f)))`

`(cons (cdr '(a b)) (cdr '(c d)))`

`(cons (cons 'a '()) (cons 'b '()))`

Define

1. Define a function using the name of another function

> **(define first car)**

> **(define rest cdr)**

> **(define second cadr)**

> **(first '(a b c))**

a

> **(rest '(a b c))**

(b c)

> **(second '(a b c))**

b

2. Define a function using a lambda-expression

Recall lambda calculus notation:

$$\text{double}(x) \equiv \lambda x (2 * x)$$

$$f(x, y) \equiv \lambda xy (x^2 + y^2)$$

```
> (define double  
  (lambda (x) (* 2 x)))
```

```
> (define f  
  (lambda (x y)  
    (+ (* x x) (* y y))))
```

```
> (double 6)
```

```
12
```

```
> (f 2 3)
```

```
13
```

3. Define a function using a shorthand version of #2 (most common form)

```
(define (double x)  
  (* 2 x))
```

```
(define (f x y)  
  (+ (* x x) (* y y)))
```

Note: The body of the function can contain a sequence of expressions. The value of the *last* expression is the return value of the function.

Thus,

```
(define (f x y)  
  (* x x)  
  (* y y)  
  (+ x y) )
```

isn't useful, but

```
(define (f x y)  
  (display x) (newline)  
  (display y) (newline)  
  (+ (* x x) (* y y)))
```

might be.

Exercise (define)

Define a function **add-firsts** which takes two lists as arguments and returns the sum of the first element of each list.

```
> (add-firsts '(2 3) '(10 50))  
12
```

Branching: cond

Form:

(cond (*predicate*₁ *expression*₁)
 (*predicate*₂ *expression*₂)
 (*predicate*₃ *expression*₃)
 ...
 (*predicate*_{*n*} *expression*_{*n*}))

Meaning:

Evaluate *predicate*₁, *predicate*₂, etc., until one predicate evaluates to true.

Return the corresponding *expression*.

Undefined if no *predicate* is true.

Note: As with *define*, a predicate can be followed by a sequence of expressions:

(*predicate* *expr*_{*A*} *expr*_{*B*} ... *expr*_{*last*})

If so, the value of the *last* expression is returned.

Examples (cond)

```
(define (monus x y)           ; Like minus,  
  (cond ((< x y) 0)          ; but 0 if x < y  
        (#T (- x y))))
```

```
(define (maximum x y)         ; Returns the larger  
  (cond ((> x y) x)          ; of x and y  
        (#T y)))
```

Usually, the last predicate is `#T`, to make sure one predicate is true. Scheme allows *else* here:

```
(define (maximum x y)         ; Returns the larger  
  (cond ((> x y) x)          ; of x and y  
        (else y)))
```

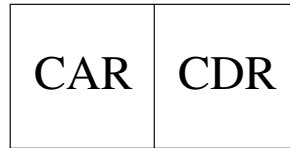
A *cond* expression can contain as many predicate-expression pairs as you want. For example:

```
(define (maximum x y)         ; Returns the larger  
  (cond ((> x y) x)          ; of x and y  
        ((= x y) x)  
        (else y)))
```

Some built-in Scheme predicates

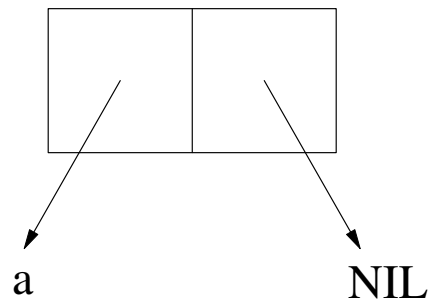
eq?	#T if symbols with same value
eqv?	#T if atoms with same value
equal?	#T if objects with same value (includes lists)
null?	#T if empty list
number?, string?, symbol?, atom? list?	#T if argument has specified type
=, <, <=, >, >=	Mathematical comparisons
and, or, not	Logical operators

List Implementation

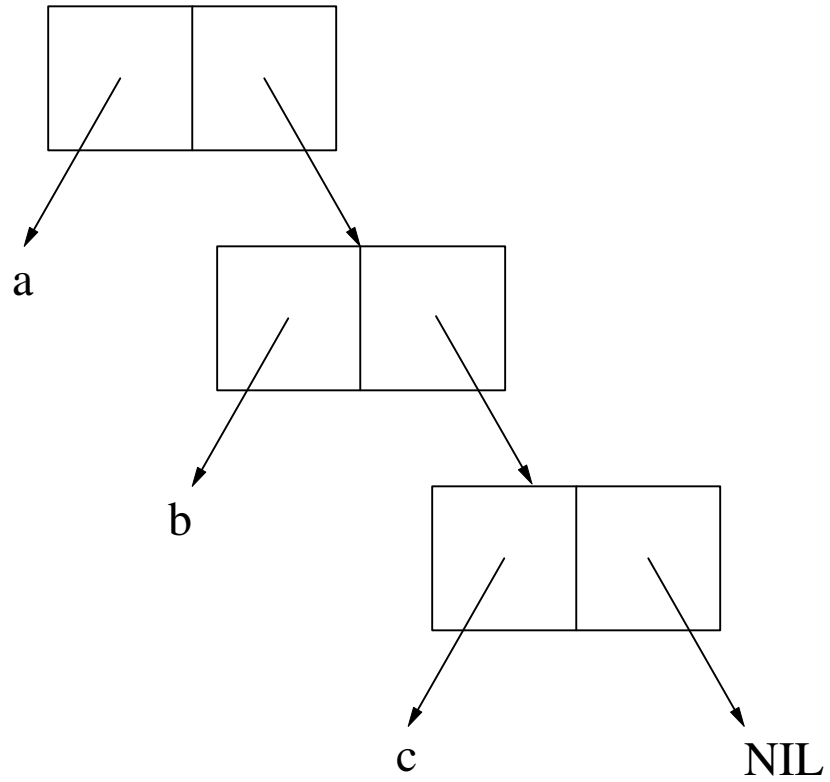


Linked structure of 2-part cells

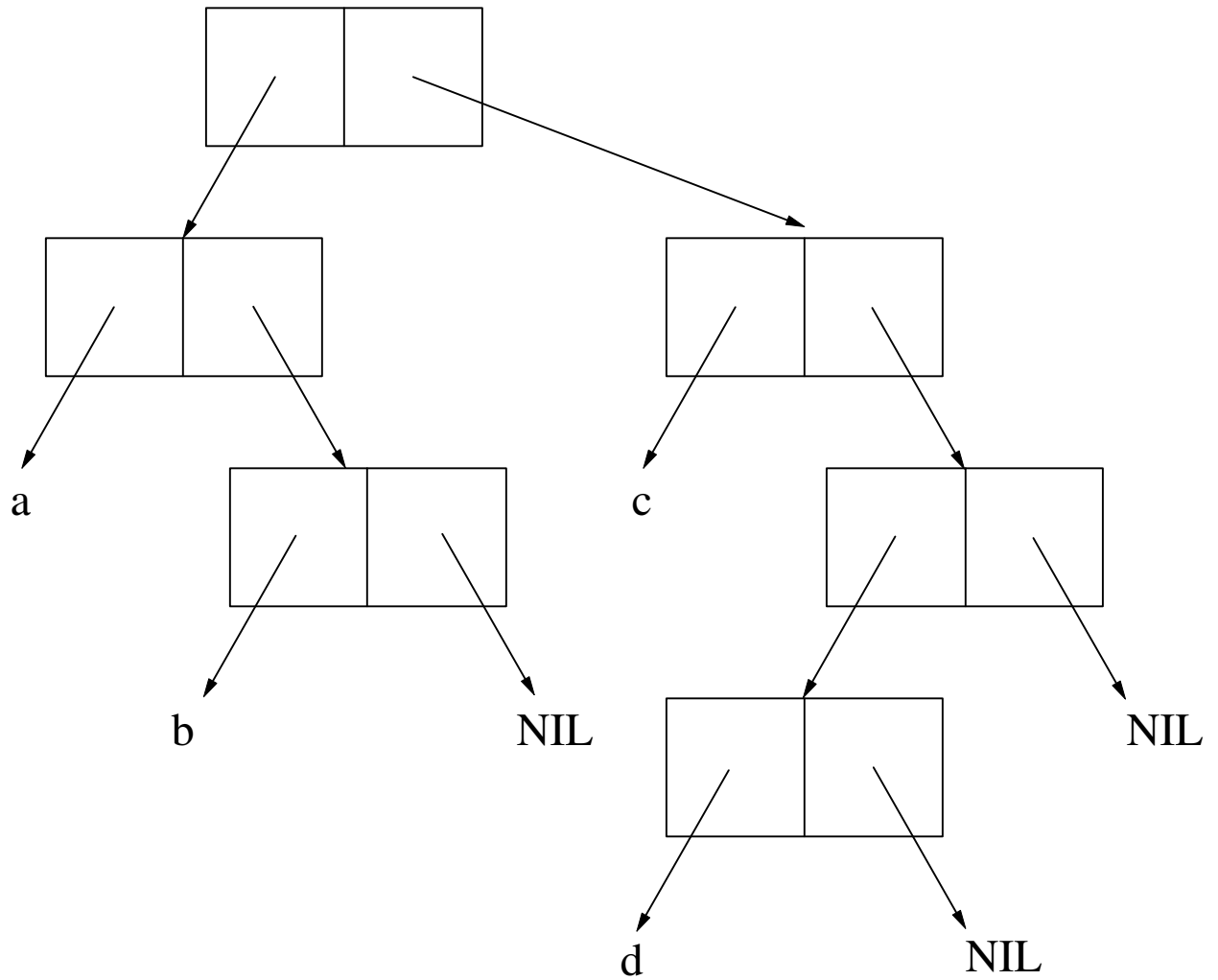
(a)



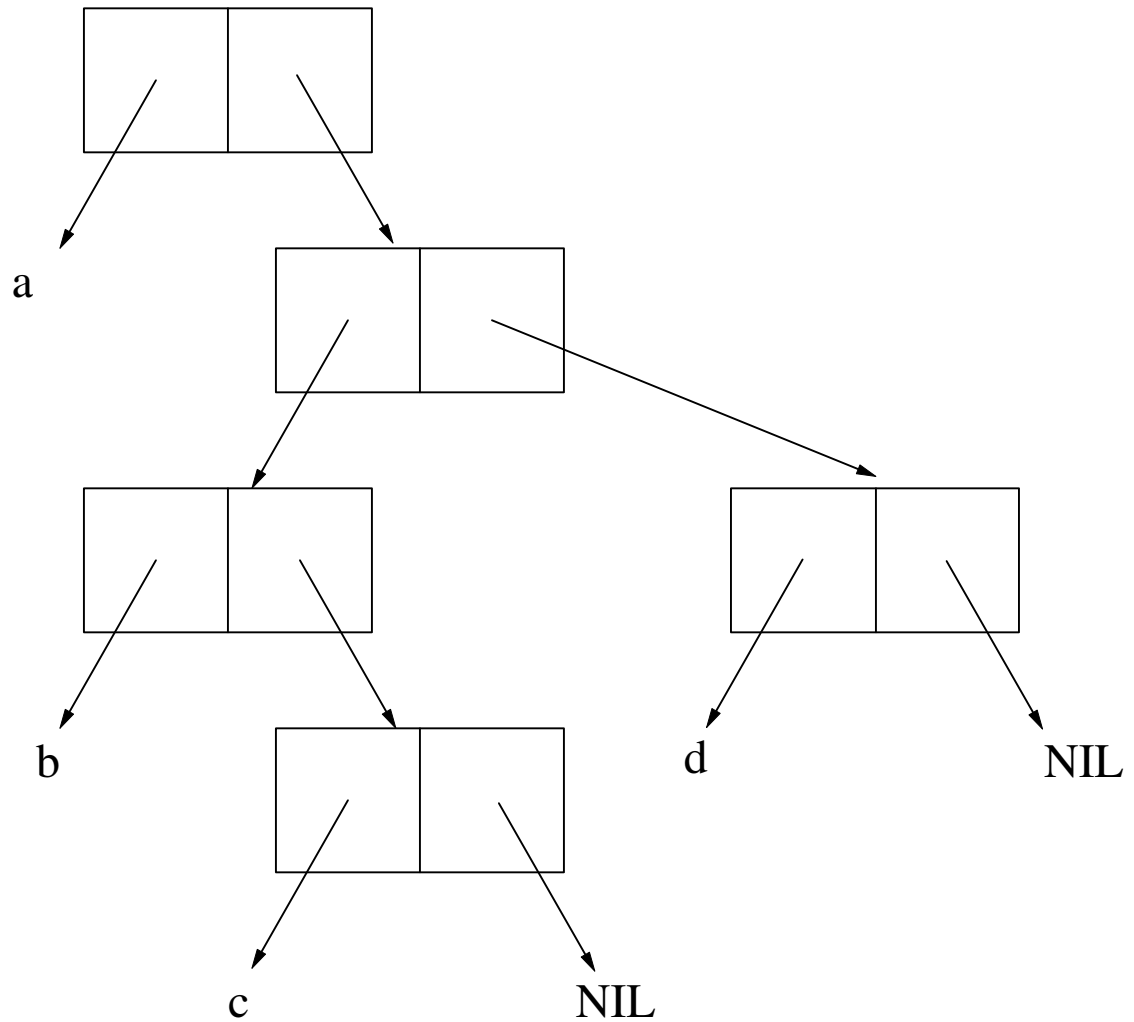
(a b c)



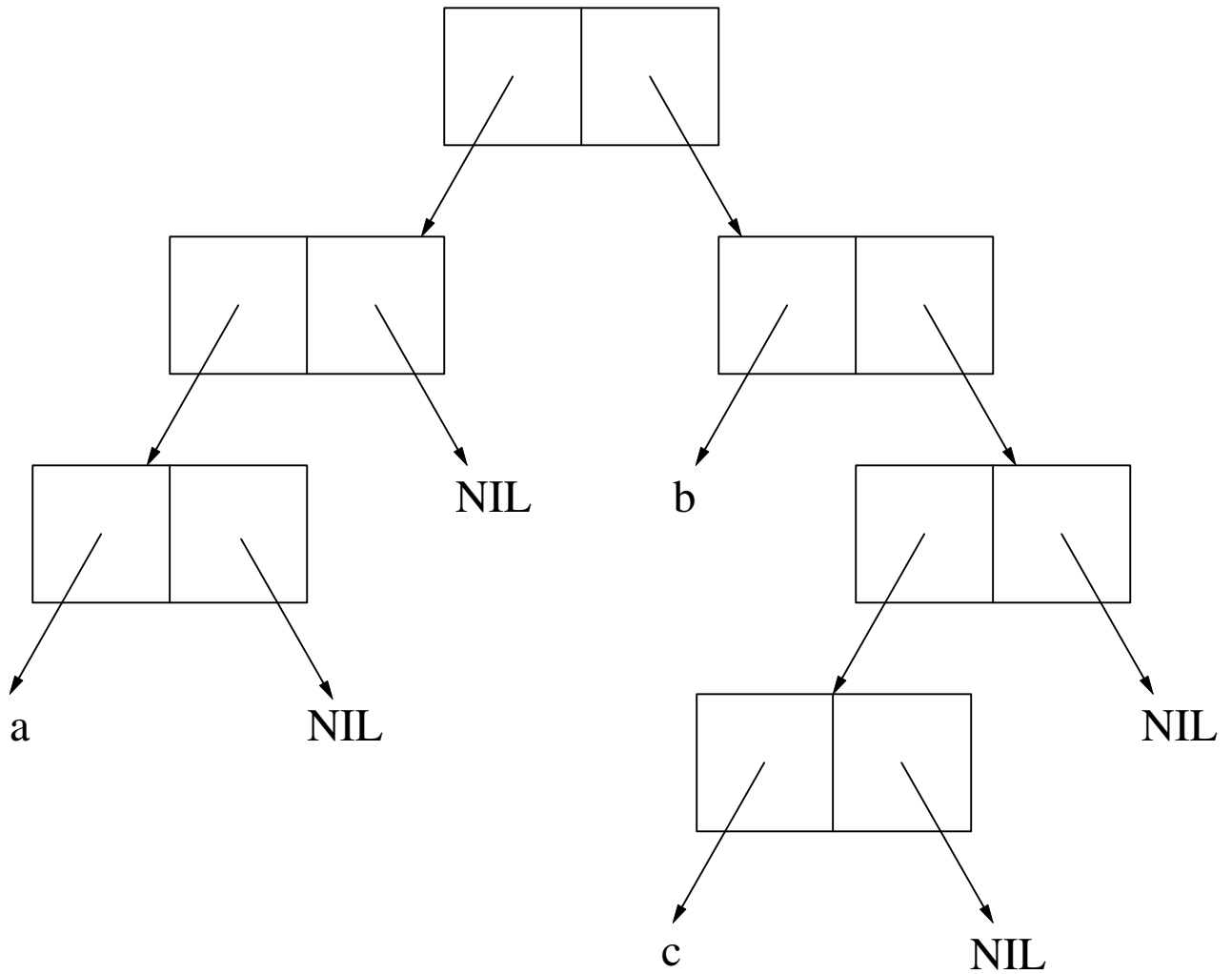
((a b) c (d))



What list corresponds to this structure?

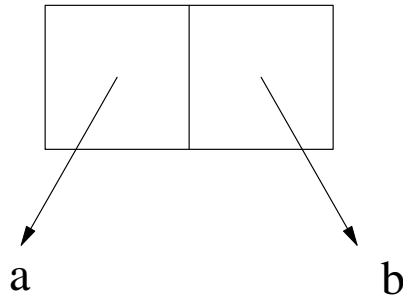


What list corresponds to this structure?

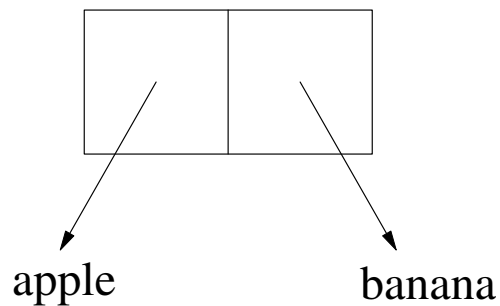


Pairs ("Dotted pairs")

(a . b)



(apple . banana)



A list is a special kind of pair in which the rightmost pointer is nil.

(apple) is **(apple . ())**

(a b c) is **(a . (b . (c . ()))**

Using recursion in Scheme

Scheme has a simple counted loop, but most repetition is done with **recursion**.

Examples:

```
(define (sumlist L)
  ; Returns the sum of the numbers in L
  (cond ((null? L) 0)
        (else (+ (car L) (sumlist (cdr L))))))
```

```
(define (add_pairs x y)
  ; Return a list containing the sums of the
  ; numbers in x and y:
  ; (add_pairs '(1 3 5) '(2 10 20)) returns (3 13 25).
  ; Assume the lists have the same length.
  (cond ((null? x) '())
        (else (cons (+ (car x) (car y))
                      (add_pairs (cdr x) (cdr y))))))
```